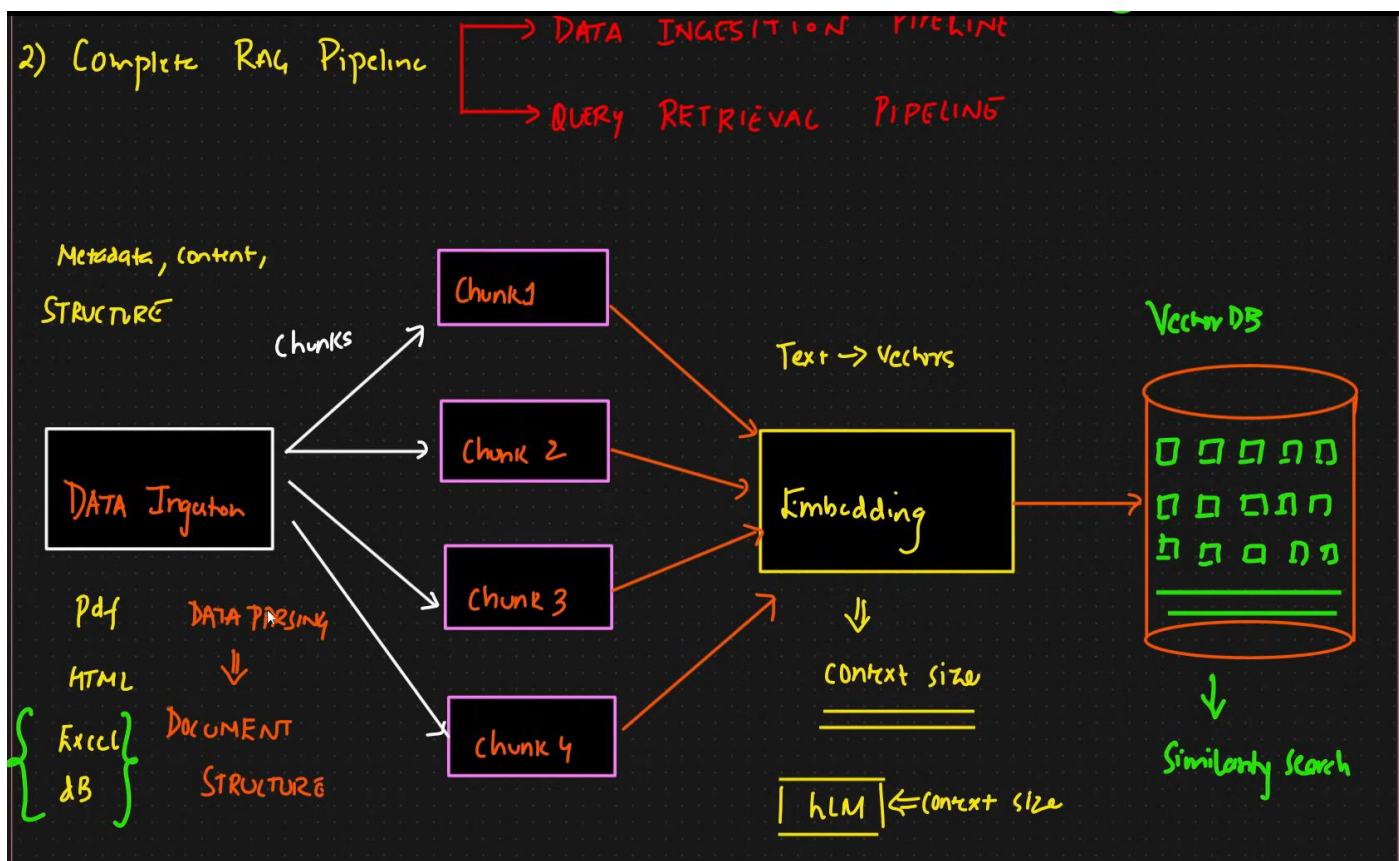
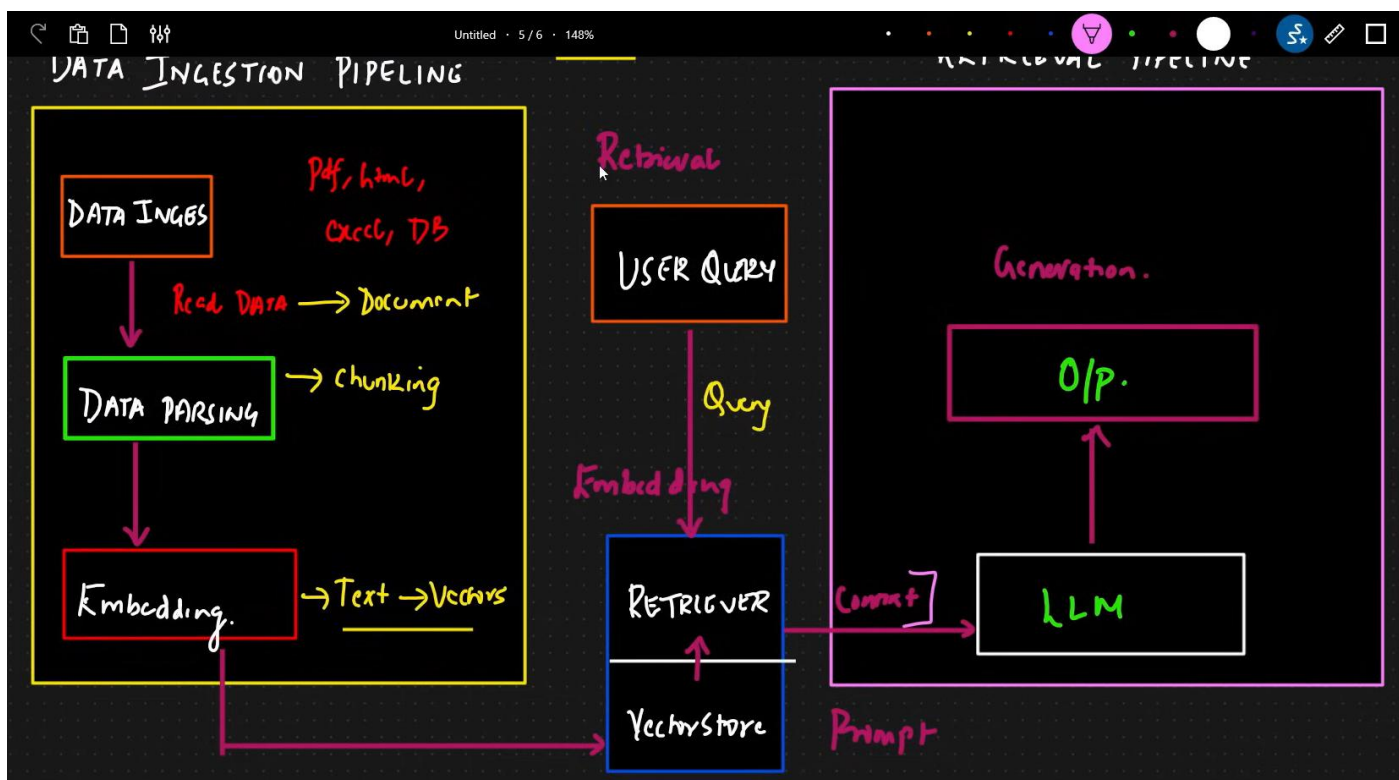




Great diagrams! These are from a solid RAG tutorial. Let me build a full visual explainer of the complete workflow based on what you've drawn — both pipelines. Here's the full breakdown of how it works — every node is clickable for a deeper dive.

RAG = Retrieval-Augmented Generation. It has two distinct pipelines, both shown in your diagrams.

Phase 1 — Data Ingestion (offline, runs once)

- 1. Data Sources** — You start with raw documents: PDFs, HTML pages, Excel files, databases. These are your "knowledge base" that the LLM doesn't know about natively.
 - 2. Data Parsing** — Each document gets parsed to extract its structure — headings, paragraphs, tables, metadata. Libraries like `PyMuPDF` (for PDF) or `UnstructuredIO` handle this. The output is a clean `Document` object with content + metadata.
 - 3. Chunking** — The parsed document is split into smaller pieces called chunks (Chunk 1, 2, 3, 4... in your diagram). This is critical because LLMs have a limited context window — you can't feed a 100-page PDF in one shot. Typical chunk size: 300–500 tokens with some overlap between adjacent chunks so context isn't lost at boundaries.
 - 4. Embedding** — Each chunk is passed through an **embedding model** (e.g. `text-embedding-ada-002` from OpenAI, or `BAAI/bge-base`) which converts the text into a high-dimensional float vector (e.g. 1536 numbers). These vectors capture semantic meaning — similar text lands close together in vector space.
 - 5. Vector DB** — All chunk vectors are stored in a vector database (FAISS, Pinecone, MongoDB Atlas, Chroma). Each vector is indexed alongside the original chunk text and its metadata. This is your searchable knowledge store.
-

Phase 2 — Query Retrieval (online, every request)

- 1. User Query** — A user asks a natural language question.
 - 2. Embed Query** — The question is embedded using the **exact same model** used during ingestion. This is essential — both the query and the stored chunks must live in the same vector space for similarity to make sense.
 - 3. Similarity Search (Vector DB → Retriever)** — The query vector is compared against all stored chunk vectors using cosine similarity or dot product. The top-k most similar chunks (e.g. top 3–5) are retrieved. This is the "R" in RAG.
 - 4. LLM + Context Window** — The retrieved chunks are injected into a prompt alongside the user's question. The prompt looks roughly like: *"Here is relevant context: [chunk1] [chunk2] [chunk3]... Answer this question: [user query]"*. The context size (shown in your Image 2) limits how many chunks you can pass — this is why chunking strategy matters so much.
 - 5. Output** — The LLM generates a response grounded in the retrieved chunks, not just its training data. This eliminates hallucinations on domain-specific knowledge and makes the answer traceable to a source.
-

The key insight from your diagram: **Phase 1 and Phase 2 share the same embedding model** — that's the bridge between ingestion and retrieval. If they differ, the similarity search breaks completely.

In your pipeline, an **embedding model** is the component that converts human-readable text into a numerical form that a machine can compare and search efficiently.

What it actually means

An embedding model takes a piece of text (like one of your chunks) and turns it into a **vector**—a list of numbers such as:

"The cat sat on the mat" → [0.12, -0.87, 0.43, ..., 0.05]

These numbers aren't random—they encode the **meaning** of the text.

Why this matters

The key idea is based on a concept from vector space model:

- Texts with **similar meaning** end up **close together** in this high-dimensional space
- Texts with **different meaning** are **far apart**

So for example:

- "How to reset my password"
- "I forgot my login credentials"

→ These will have **similar embeddings**, even though the wording is different.

Role in your pipeline

In your Phase 1:

1. You chunk the document
2. Each chunk goes into an embedding model
3. The model outputs a vector for each chunk
4. These vectors are stored in your vector database

Later (during querying):

- The user's question is also embedded
- You compare its vector to stored vectors
- You retrieve the **most similar chunks**

First: The Problem

In normal RAG, you use **semantic (dense) search**:

- It understands *meaning*
- But sometimes **misses exact words**

Example:

- Query: **"What is LK-99?"**

Problem:

- “LK-99” is a **weird, specific keyword**
 - A semantic model may not understand it
 - So it might **fail to retrieve the right chunk**
-

The Two Search Types

1. BM25 (Keyword Search)

Based on the classic BM25 algorithm

- Looks for **exact words**
- Like Ctrl + F on steroids

☒ Good at:

- Rare terms (like “LK-99”)
- Exact matches

☒ Bad at:

- Understanding meaning
-

2. Dense Search (Embedding Search)

Based on embeddings (semantic meaning)

- Finds **similar meaning**, not exact words

☒ Good at:

- “How to reset password” $\approx$ “forgot login credentials”

☒ Bad at:

- Strange terms like “LK-99”
-

Hybrid Search = Best of Both

Hybrid search =

 **BM25 (keywords) + Dense (meaning)**

You run **both searches in parallel**.

Example

Your documents:

1. “LK-99 is a room temperature superconductor candidate.”
 2. “Superconductors conduct electricity with no resistance.”
 3. “How to reset your password in the system.”
-

User query:

 “What is LK-99?”

Step 1: BM25 results (keyword match)

1. “LK-99 is a room temperature superconductor candidate.” ☒ (perfect match)
 2. Others → low relevance
-

Step 2: Dense results (semantic)

1. “Superconductors conduct electricity...” (related concept)
 2. Maybe misses the exact LK-99 chunk ✗
-

Step 3: Merge Results

You combine both lists using
Reciprocal Rank Fusion (RRF)

Don’t worry about the formula—just know:

 It rewards chunks that rank high in either or both lists

Final Result

Top result becomes:

- “LK-99 is a room temperature superconductor candidate.” ☒

I'll keep this **simple, practical, and example-driven**—like how you'd explain in an interview.

2 Reranking (simple explanation)

Idea

Initial search gives you “**roughly good**” results.
Reranking makes them “**actually the best**”.

Step-by-step

Step 1: You retrieve top 10 chunks

Using embeddings (fast but not perfect)

Example query:

 “What is LK-99?”

Top 10 results might include:

- Chunk A: “LK-99 is a superconductor candidate” ☒
 - Chunk B: “Superconductors reduce resistance”
 - Chunk C: “Battery materials research”
 - Some irrelevant ones ✗
-

Step 2: Reranking (the smart judge)

Now you use a **cross-encoder model**:

- It reads **query + chunk together**
- Gives a **precise relevance score**

Unlike embedding models:

- Embedding → compares separately (less accurate)
 - Cross-encoder → compares together (more accurate)
-

Step 3: Reorder results

Now ranking becomes:

1. “LK-99 is a superconductor candidate” ☒
2. “Superconductors reduce resistance”
3. Others...

You keep only **top 3–5** and send to LLM.

Simple analogy

- Retrieval = shortlist candidates
 - Reranking = final interview round
-

Tradeoff

- ☒ Much more accurate
 - ☒ Slower (can't precompute)
-
-

3 Chunking Strategies (super simple)

Why chunking?

LLM can't read huge documents → break into pieces

Types

1. Fixed-size chunking

Split every 500 tokens

☒ Problem:
Cuts sentences randomly

Example:

“The refund policy states that customers...”
→ cut in middle 😞

2. Recursive chunking (best default)

Splits like:

- Paragraph → Sentence → Words

Used in tools like LangChain

- ☒ Keeps meaning intact
-

3. Semantic chunking

Based on meaning (topic changes)

Example:

- Chunk 1: Refund policy
- Chunk 2: Shipping policy

Uses embeddings to detect shift

- ☒ Most accurate
 - ☒ Slow & complex
-



Rule of thumb

- Small chunks (150–300) → precise answers
 - Big chunks (500–1000) → more context
 - Always add overlap (10–15%)
-
-

4 Metadata Filtering (very easy)



Idea

Filter **before** searching



Example

User asks:

👉 “Refund policy for 2024”

Instead of searching everything:

First filter:

- year = 2024
- type = policy

Then search inside that

Why important?

- Faster ⚡
- More accurate 

Analogy

Like searching emails:

- First filter: “from boss + last month”
- Then read

5 RAG vs Fine-tuning (INTERVIEW GOLD)

Core difference

RAG (Retrieval-Augmented Generation)

- Model stays same
- You **give it documents at runtime**

☒ Best for:

- Changing data (docs, policies, news)
- Need citations

Fine-tuning

- You **train the model itself**

☒ Best for:

- Style (“talk formally”)
- Format (“always return JSON”)

Key insight (VERY important)

 RAG solves **knowledge problem**

 Fine-tuning solves **behavior problem**

Example

- “What is our refund policy?”
→ Use RAG (data changes)
- “Always respond like a lawyer”
→ Fine-tuning

Pro setup

 Combine both

6 Hallucination Reduction (simple + practical)

Problem

LLM makes up answers when:

- Data missing
 - Retrieval is bad
-

Solutions

1. Prompting

Tell model clearly:

 “Answer only from context. If not found, say ‘I don’t know’.”

This alone helps a LOT.

2. Better retrieval

- Hybrid search
- Reranking
- Good chunking

 Better input = better output

3. Faithfulness check

Use another LLM to verify answer

Related to RAGAS

It checks:

- “Is this answer supported by the chunks?”
-

4. Fix chunking gaps

If info is split badly:

- Model won't find it → hallucinates

Solution:

- Overlap chunks
 - Better splitting
-

Simple analogy

Hallucination = student guessing in exam

Fix = give better notes + tell them not to guess

One-line summary (great for interviews)

- **Reranking** → improves result quality after retrieval
 - **Chunking** → controls how knowledge is split
 - **Metadata filtering** → narrows search space
 - **RAG vs Fine-tuning** → knowledge vs behavior
 - **Hallucination reduction** → better retrieval + strict prompting
-

If you want, I can turn this into a **perfect interview answer script (2–3 min)** or **system design diagram**.

Jupiter vs Fast API

FASTAPI - Production RAG with settings from Jupyter experiments

from openai import OpenAI

import numpy as np

from sklearn.metrics.pairwise import cosine_similarity

class RAGService:

def __init__(self):

self.client = OpenAI()

USE SETTINGS WE DISCOVERED IN JUPYTER:

self.chunk_size = 1000 # ← From Experiment 1

self.embedding_model = "text-embedding-ada-002" # ← From Experiment 2

self.similarity_method = cosine_similarity # ← From Experiment 3

def process_document(self, file_path: str):

"""Production RAG pipeline"""

Extract text

text = self.extract_text(file_path)

Chunk (using optimal size from Jupyter)

chunks = self.chunk_text(text, self.chunk_size)

Create embeddings (using optimal model from Jupyter)

embeddings = self.create_embeddings(chunks)

Store in database

self.save_to_db(chunks, embeddings)

return {"chunks": len(chunks), "status": "success"}

def retrieve_relevant_chunks(self, query: str, top_k: int = 3):

"""Production retrieval (using optimal method from Jupyter)"""

Get query embedding

query_embedding = self.create_embeddings([query])[0]

```

# Load all document embeddings from DB

doc_embeddings = self.load_embeddings_from_db()


# Calculate similarity (using method from Jupyter)

similarities = self.similarity_method([query_embedding], doc_embeddings)[0]


# Get top matches

top_indices = np.argsort(similarities)[-top_k][::-1]


return self.get_chunks_by_indices(top_indices)
...

```

📁 ****THE COMPLETE WORKFLOW****

****1. RESEARCH PHASE (Jupyter)**** 📊

...

Jupyter Notebook (One-time experiments)



Test 5 different chunk sizes

Test 3 different embedding models

Test 3 different similarity methods



FIND OPTIMAL SETTINGS:

✓ Chunk size: 1000

✓ Model: text-embedding-ada-002

✓ Method: Cosine similarity

✓ Accuracy: 89%

...

****2. PRODUCTION PHASE (FastAPI)**** 🚀

...

FastAPI (Ongoing production use)



Implement RAG with optimal settings

Create API endpoints

Connect to database

Add authentication

Deploy to cloud



SERVE TO USERS 24/7:

✓ /documents/upload endpoint

✓ Process user documents

✓ Answer user questions

✓ 1000+ users can use it

...

📅 ****REAL PROJECT TIMELINE****

****Week 1-2: Research (Jupyter)****

...

Day 1-3: Test chunk sizes → Result: 1000 is best

Day 4-6: Test embedding models → Result: ada-002 is best

Day 7-10: Test retrieval methods → Result: cosine is best

Day 11-14: Optimize prompts → Result: "You are helpful" works

...

****Week 3-8: Production (FastAPI + React)****

...

Week 3: Build RAG service with optimal settings

Week 4: Create API endpoints

Week 5: Build frontend UI

Week 6: Add authentication

Week 7: Connect database

Week 8: Deploy and test

...

📄 ****THE ANALOGY****

****JUPYTER = RECIPE TESTING KITCHEN**** 🧑🍳

...

Try 10 different recipes:

- Recipe A: 72% delicious
- Recipe B: 89% delicious ← WINNER!
- Recipe C: 81% delicious

Experiment with ingredients:

- Salt vs Sugar
- Butter vs Oil
- Bake 350°F vs 375°F

RESULT: Recipe B at 375°F with butter = BEST!

...

****FASTAPI = RESTAURANT KITCHEN**** 🍷

...

Cook Recipe B (the winner) for customers:

- Customer 1 orders → Cook Recipe B
- Customer 2 orders → Cook Recipe B
- Customer 100 orders → Cook Recipe B

Serve 1000+ customers per day

Quality control

Consistent results

Professional service

...

💡 ****WHY WE DON'T USE JUPYTER FOR PRODUCTION RAG****

****Jupyter CAN'T:****

- ❌ Serve multiple users simultaneously
- ❌ Provide API endpoints
- ❌ Integrate with authentication
- ❌ Connect to production database
- ❌ Auto-restart on crash
- ❌ Scale to 1000+ users
- ❌ Be deployed to cloud easily
- ❌ Handle concurrent requests

****FastAPI CAN:****

- ✅ Serve 1000+ users concurrently
- ✅ Provide REST API endpoints
- ✅ Integrate with JWT auth
- ✅ Connect to PostgreSQL
- ✅ Auto-restart with uvicorn
- ✅ Scale horizontally
- ✅ Deploy to Railway/AWS/etc
- ✅ Handle 100 requests/second

🔄 ****THE KEY DIFFERENCE****

Aspect	Jupyter (Research)	FastAPI (Production)
-----	-----	-----

Purpose	Find best approach	Serve the best approach
Frequency	One-time experiments	24/7 operation
Users	1 researcher	1000+ end users
Speed	Doesn't matter	Must be fast
Reliability	Can crash, restart	Must be stable
Integration	Standalone	Connected to auth, DB, UI

📝 **WHAT WE DID IN THIS PROJECT**

We SKIPPED the Jupyter phase! ⚡

Why?

- Used **standard best practices** from research:
 - Chunk size: 1000 (industry standard)
 - Embedding: ada-002 (OpenAI recommended)
 - Similarity: Cosine (most common)

If this was a research project:

...

Week 1-2: Test in Jupyter (find optimal settings)

Week 3-8: Build in FastAPI (use optimal settings)

...

What we actually did:

...

Week 1-8: Build in FastAPI directly (use standard settings)

LangChain Document Structure

Understanding the core components of LangChain Documents



LangChain Document

```
from langchain.schema import Document
```

Core Components:

- **page_content (str)**
- **metadata (dict)**

Creating a Document

```
doc = Document(  
    page_content= "RAG is a technique..." ,  
    metadata={  
        "source" : "chapter1.pdf" ,  
        "page" : 5 ,  
        "timestamp" : "2024-01-15"  
    }  
)
```

page_content (String)

The actual text content of the document

- Contains the main information to be embedded and searched
- Must be a string (can be any length)

Examples:

Research Paper:

"Retrieval-Augmented Generation (RAG) combines the benefits of pre-trained language models with information retrieval systems to generate more accurate and contextual responses..."

Product Manual:

"To install the software, first ensure your system meets the minimum requirements: Windows 10 or later, 8GB RAM, and at least 20GB of free disk space..."

Best Practices:

- Keep content focused and coherent
- Remove unnecessary formatting before storage
- Consider chunk size limits (typically 500-2000 tokens)

metadata (Dictionary)

Additional information about the document

- Used for filtering, tracking, and context
- Can contain any JSON-serializable data

Common Metadata Fields:

source

File path or URL

"docs/manual.pdf"

page / chunk_id

Location in document

page: 42, chunk: 7

timestamp

Creation/modification date

"2024-01-15T10:30:00Z"

author

Document creator

"John Doe"

category / type

Document classification

"technical", "legal"

language

Content language

"en", "es", "fr"

Tip: Add custom fields for your use case

Examples: department, security_level, version, keywords, embeddings_model



LangChain Document Loaders

PDFLoader

```
from langchain.document_loaders import PyPDFLoader  
loader = PyPDFLoader("file.pdf")  
documents = loader.load()
```

CSVLoader

```
from langchain.document_loaders import CSVLoader  
loader = CSVLoader("data.csv")  
documents = loader.load()
```

WebBaseLoader

```
from langchain.document_loaders import WebBaseLoader  
loader = WebBaseLoader("https://...")  
documents = loader.load()
```

DirectoryLoader

```
from langchain.document_loaders import DirectoryLoader  
loader = DirectoryLoader("./docs")  
documents = loader.load()
```

Additional Loaders:

- UnstructuredLoader (multiple formats)
- NotionDirectoryLoader
- S3FileLoader
- ConfluenceLoader
- JSONLoader
- GoogleDriveLoader
- YouTubeLoader
- DocugamiLoader
- TextLoader
- AirtableLoader
- WikipediaLoader
- EverNoteLoader
- GitbookLoader
- SlackDirectoryLoader
- ArxivLoader
- HuggingFaceDatasetLoader

Loading pdf file into documents:-

```
requirements.txt U document.ipynb U python_intro.txt U 1-langchain-document-components.svg
notebook > document.ipynb > Data Ingestion > from langchain_community.document_loaders import PyPDFLoader, PyMuPDFLoader
+ Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables YTRAG (Python 3.13.2)
## load all the text files from the directory
dir_loader=DirectoryLoader(
    ..../data/text_files",
    glob="**/*.txt", ## Pattern to match files
    loader_cls= TextLoader, ##loader class to use
    loader_kwargs={'encoding': 'utf-8'},
    show_progress=False
)

documents=dir_loader.load()
documents

[12] ✓ 0.0s Python
... [Document(metadata={'source': '..\\data\\text_files\\machine_learning.txt'}, page_content='Machine Learning
Document(metadata={'source': '..\\data\\text_files\\python_intro.txt'}, page_content='Python Programming
```

```
from langchain_community.document_loaders import PyPDFLoader, PyMuPDFLoader

## load all the text files from the directory
dir_loader=DirectoryLoader(
    "../data/pdf",
    glob="**/*.pdf", ## Pattern to match files
    loader_cls= PyMuPDFLoader, ##loader class to use
    show_progress=False
)

pdf_documents=dir_loader.load()
pdf_documents

✓ 0.4s Python
[Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2024
Document(metadata={'producer': 'pdfTeX-1.40.25', 'creator': 'LaTeX with hyperref', 'creationdate': '2024
```

embedding And vectorStoreDB

```
import numpy as np
from sentence_transformers import SentenceTransformer
import chromadb
from chromadb.config import Settings
import uuid
from typing import List, Dict, Any, Tuple
from sklearn.metrics.pairwise import cosine_similarity
```

```
class EmbeddingManager:
    """Handles document embedding generation using SentenceTransformer"""

    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        """
        Initialize the embedding manager

        Args:
            model_name: HuggingFace model name for sentence embeddings
        """
        self.model_name = model_name
        self.model = None
        self._load_model()
```

```
def _load_model(self):
    """Load the SentenceTransformer model"""
    try:
        print(f"Loading embedding model: {self.model_name}")
        self.model = SentenceTransformer(self.model_name)
        print(f"Model loaded successfully. Embedding dimension: {self.model.get_sentence_embedding_dimension()}")
    except Exception as e:
        print(f"Error loading model {self.model_name}: {e}")
        raise
```

```

def generate_embeddings(self, texts: List[str]) -> np.ndarray:
    """
    Generate embeddings for a list of texts

    Args:
        texts: List of text strings to embed

    Returns:
        numpy array of embeddings with shape (len(texts), embedding_dim)
    """
    if not self.model:
        raise ValueError("Model not loaded")

    print(f"Generating embeddings for {len(texts)} texts...")
    embeddings = self.model.encode(texts, show_progress_bar=True)
    print(f"Generated embeddings with shape: {embeddings.shape}")
    return embeddings

```

```

## initialize the embedding manager

```

```

embedding_manager=EmbeddingManager()
embedding_manager

```

✓ 4.7s

Loading embedding model: all-MiniLM-L6-v2

Model loaded successfully. Embedding dimension: 384

VectorStore

```
class VectorStore:
    """Manages document embeddings in a ChromaDB vector store"""

    def __init__(self, collection_name: str = "pdf_documents", persist_directory: str = "../data/vector_store"):
        """
        Initialize the vector store

        Args:
            collection_name: Name of the ChromaDB collection
            persist_directory: Directory to persist the vector store
        """
        self.collection_name = collection_name
        self.persist_directory = persist_directory
        self.client = None
        self.collection = None
        self._initialize_store()
```

pdf_loader.ipynb > VectorStore > class VectorStore:

+ Markdown | ▶ Run All ↺ Restart ⌵ Clear All Outputs | (x) Jupyter Variables ≡ Outline ...

```
    def _store(self):
        """Initialize ChromaDB client and collection"""
        try:
            # Create persistent ChromaDB client
            os.makedirs(self.persist_directory, exist_ok=True)
            self.client = chromadb.PersistentClient(path=self.persist_directory)

            # Get or create collection
            self.collection = self.client.get_or_create_collection(
                name=self.collection_name,
                metadata={"description": "PDF document embeddings for RAG"}
            )
            print(f"Vector store initialized. Collection: {self.collection_name}")
            print(f"Existing documents in collection: {self.collection.count()}")

        except Exception as e:
            print(f"Error initializing vector store: {e}")
            raise
```

chunks|

⊗ 0.2s

```
-----
NameError                                Traceback (most recent call last)
Cell In[10], line 1
----> 1 split_chunks

NameError: name 'split_chunks' is not defined
```

```
### Convert the text to embeddings
texts=[doc.page_content for doc in chunks]
texts
```

✓ 0.0s Python

['Provided proper attribution is provided, Google hereby grants permission to\nreproduce the tables and figures in this paper sol
'mechanism. We propose a new simple network architecture, the Transformer,\nbased solely on attention mechanisms, dispensing with
'best models from the literature. We show that the Transformer generalizes well to\nother tasks by applying it successfully to E
'efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and\nimplementing
'1 Introduction\nRecurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks\nin particular, I
'sequential nature precludes parallelization within training examples, which becomes critical at longer\nsequence lengths, as me
'In this work we propose the Transformer, a model architecture eschewing recurrence and instead\nrelying entirely on an attention
'in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes\nit more difficult to learn
'textual entailment and learning task-independent sentence representations [4, 27, 28, 22].\nEnd-to-end memory networks are base
'Here, the encoder maps an input sequence of symbol representations (x1, ..., xn) to a sequence\nof continuous representations
'Figure 1: The Transformer - model architecture.\nThe Transformer follows this overall architecture using stacked self-attention
'itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding\nlayers, produce output
'3.2 Attention\nAn attention function can be described as mapping a query and a set of key-value pairs to an output, weighing the
'Scaled Dot-Product Attention\nMulti-Head Attention\nFigure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention
into a matrix Q. The keys and values are also packed together into matrices K and V. We compute\nthe matrix of outputs as:\nAt
'dot product attention without scaling for larger values of dk [3]. We suspect that for large values of\ndk, the dot products grow

```
### Convert the text to embeddings
texts=[doc.page_content for doc in chunks]

## Generate the Embeddings

embeddings=embedding_manager.generate_embeddings(texts)

##store int he vector database
vectorstore.add_documents(chunks,embeddings)
```

✓ 8.2s Python

Generating embeddings for 359 texts...
Batches: 100%|██████████| 12/12 [00:07<00:00, 1.66it/s]
Generated embeddings with shape: (359, 384)
Adding 359 documents to vector store...
Successfully added 359 documents to vector store
Total documents in collection: 359

OPEN EDITORS

requirements.txt

1-langchain-docu...

pdf_loader.ipynb...

YTRAG

.venv

data

pdf

attention.pdf

embedding...

objectdetection.pdf

proposal.pdf

text_files

vector_store

notebook

1-langchain-docume...

document.ipynb

pdf_loader.ipynb

.gitignore

notebook > pdf_loader.ipynb > Retriever Pipeline From VectorStore > rag_retriever.retrieve("What is attention is all you need")

+ Code + Markdown | ▶ Run All ↺ Restart ≡ Clear All Outputs | Jupyter Variables ... YTRAG (Python)

```
print(f"Error during retrieval: {e}")
return []

rag_retriever=RAGRetriever(vectorstore,embedding_manager)
```

[16] ✓ 0.0s

```
rag_retriever
```

[17] ✓ 0.0s

... <__main__.RAGRetriever at 0x2b7e77af770>

▶ ≡ ▶ ▶ ▶ ≡ ... 🗑

```
rag_retriever.retrieve("What is attention is all you need")
```

[]